

Taller 6

Procesos de usuario en EDOS

Tomás Rodeghiero

Sistemas Operativos — Universidad Nacional de Río Cuarto

2026

En este taller trabajé sobre el paso `08-process` de EDOS, que es donde el kernel pasa a poder cargar y correr programas de usuario. En este informe voy resolviendo los nueve ejercicios de la consigna: primero el booteo y la ejecución del `init`, después el análisis del allocator de páginas, el layout de memoria del kernel, la creación de procesos, cómo se compila y enlaza el código de usuario, el camino desde el scheduler hasta la primera instrucción del proceso, y por último las dos partes con código propio (provocar un page fault y agregar el syscall `time`). Todo lo probé en QEMU (`qemu-system-riscv32`) y voy pegando las salidas que me sirvieron para verificar cada punto.

Índice

1. Booteo de EDOS y ejecución del <code>init</code>	1
2. Funcionamiento del <code>physical pages allocator</code>	2
3. Layout del espacio de direcciones del kernel	3
4. Pasos de <code>create_process()</code> en <code>task.c</code>	4
5. <code>user.ld</code> , entry point y <code>mkefs.sh</code>	5
6. Del scheduler a la primera instrucción del usuario	6
7. Provocar un page fault en <code>init.c</code>	7
8. Camino completo del syscall <code>getpid()</code>	7
9. Agregar el syscall <code>time()</code>	8

1. Booteo de EDOS y ejecución del `init`

Compilé el paso `08-process` tal cual viene y lo lancé en QEMU con:

```
qemu-system-riscv32 -machine virt -bios none -nographic \
                    -smp 1 -kernel kernel
```

La salida del booteo, ya con paginación habilitada y el primer proceso de usuario corriendo, es la siguiente (ver `entrega/salida-original.txt`):

```
Initializing kernel block allocator...
free pages=32759, free_list=80009000, kend=80009000, mem_end=88000000
ktext_start: 80000000
```

```

kdata_start: 80003000
mapping user code va=00000000 to pa=8002e000
Running scheduler on CPU 0
EDOS init process: pid=1!
Init going to sleep for 4 ticks...
pid 1 going to sleep.
Task pid=1 awake.
init awake! Finishing...
Freeing task init resources...

```

Lo que pasa, leyendo el log de arriba para abajo:

- `kernel_main()` en CPU 0 llama a `init_kalloc()` (`kalloc.c`). El log `free pages=32759` coincide con `(128 MiB - kend)/4 KiB` y confirma que la free list quedó cargada.
- `init_vm()` arma el page table del kernel (`vm.c`, `arch.c`). `map_kernel_memory()` imprime `ktext_start` y `kdata_start`.
- `create_process("init")` en `task.c` crea la tarea de usuario, carga el binario `init` desde `efs` y mapea la página del código en `va=0 → pa=0x8002e000`.
- El scheduler arranca, selecciona el `init`, entra a `start_process()` y vuelve a modo usuario.
- `main()` del `init` imprime `"EDOS init process: pid=1!"` usando los syscalls `getpid()` y `console_puts()`.
- Después llama `sleep(4)`: el syscall pone al proceso en `WAITING` sobre la condición `&ticks`; el log `"pid 1 going to sleep."` sale desde el handler `sys_sleep` en el kernel.
- Cuatro ticks después, `inc_ticks()` en `task.c` despierta al proceso (`"Task pid=1 awake."`) y el `init` imprime `"init awake!"` y retorna de `main()`, lo que ejecuta `exit()` en `edoslib.c`.
- El scheduler ve la tarea `ZOMBIE` y `free_resources()` la libera.

2. Funcionamiento del physical pages allocator

El allocator vive en `kalloc.c/kalloc.h` y administra el rango de RAM `[__kernel_end, __mem_end)`, o sea desde donde termina la imagen del kernel hasta el tope de los 128 MiB de QEMU. La estructura central es una free list simplemente enlazada cuyos nodos viven dentro de las propias páginas libres:

```

struct link { struct link *next; };
static struct link *free_list = NULL;
static unsigned int free_pages = 0;

```

Funciones expuestas (`kalloc.h`):

```

void init_kalloc(void);
void* alloc_page(void);
void free_page(void* pa);

```

- `init_kalloc()`: recorre la RAM desde `__mem_end - PAGE_SIZE` hacia abajo en pasos de `PAGE_SIZE` hasta `__kernel_end` y llama `free_page(p)` en cada una. Eso es lo que arma la free list y fija `free_pages =` cantidad de páginas libres. En el log inicial apareció `free_pages=32759`, que son los 128 MiB menos lo que ya estaba reservado para texto, datos y stacks del kernel.
- `alloc_page()`: si la lista está vacía retorna 0. Si no, saca la cabeza, hace `memset(n, 0, PAGE_SIZE)` y la devuelve. El `memset` cumple dos roles: evita filtrar datos del kernel a la

página y deja la zona en cero, lo que es cómodo para usarla, por ejemplo, como nodo de page table o como página de stack.

- `free_page(pa)`: valida que el puntero esté alineado a 4 KiB (`page_aligned`) y dentro del rango (`__kernel_end`, `__mem_end`). Si alguna falla, `panic`. Si pasa, encadena la página en la cabeza de `free_list` e incrementa `free_pages`.

Es un allocator extremadamente simple: $O(1)$ en alloc y free, sin fragmentación externa (todas las páginas son del mismo tamaño), sin metadata aparte del puntero embebido en la página libre. La única crítica es que no está protegido por un spinlock; en este paso solo una CPU lo toca, pero en pasos siguientes habría que serializarlo.

3. Layout del espacio de direcciones del kernel

`map_kernel_memory()` en `arch.c` configura mapeos 1:1 (`va = pa`) para todo lo que el kernel necesita ver con paginación encendida. Las regiones, leídas en orden del código:

```
map_region(pgtbl, UART,          UART,          PAGE_SIZE,    R|W);
map_region(pgtbl, VIRTIO0,       VIRTIO0,       PAGE_SIZE,    R|W);
map_region(pgtbl, PLIC,          PLIC,          PLIC_SIZE,    R|W);
map_region(pgtbl, ktext_start, ktext_start, ktext_size,    R|X);
map_region(pgtbl, kdata_start, kdata_start, kdata_size,    R|W);
```

con `UART = 0x10000000`, `VIRTIO0 = 0x10001000`, `PLIC = 0x0c000000`, `PLIC_SIZE = 0x400000`, `ktext_start = __kernel_start = 0x80000000`, `ktext_size = __text_end - __kernel_start` y `kdata_size = __mem_end - __text_end`. Diagrama del layout resultante:

```
Direcciones logicas del kernel (1:1 con fisicas)

+-----+ 0x00000000
| |
| no mapeado |
| |
+-----+ 0x0c000000
| PLIC (R/W) | 4 MiB (PLIC_SIZE)
+-----+ 0x0c400000
| |
| no mapeado |
| |
+-----+ 0x10000000
| UART (R/W) | 1 pagina (4 KiB)
+-----+ 0x10001000
| VIRTIO0 (R/W) | 1 pagina
+-----+ 0x10002000
| |
| no mapeado |
| |
+-----+ 0x80000000 __kernel_start
| kernel .text (R/X) |
+-----+ __text_end (en mi corrida 0x80003000)
| .rodata, .data, .bss, |
| stacks por CPU y resto |
| de la RAM (R/W) |
+-----+ 0x88000000 __mem_end (128 MiB)
```

Notas:

- Los MMIO se mapean como R/W para que el kernel pueda escribir en UART, VIRTIO y PLIC. Nunca llevan `PAGE_X`.

- La separación `ktext` (R/X) vs `kdata` (R/W) impide que el kernel ejecute datos por error.
- Como los mapeos son 1:1, una “dirección lógica” del kernel se interpreta igual con paginación encendida o apagada, lo que simplifica `enable_paging()` (basta cargar `satp` con el page table; ningún símbolo cambia de dirección).
- Cada page table del proceso copia este page table del kernel como base y le agrega los mapeos U-mode en `[0, PROC_MAX_VA)`, cosa que se ve en `exec()` (`task.c`).

4. Pasos de `create__process()` en `task.c`

`create_process(path)` es el camino completo “de la nada a una tarea de usuario RUNNABLE”. En orden:

1. `create_task(path, start_process)`:
 - Busca el primer slot `UNUSED` en `tasks[]` (con lock).
 - Inicializa el contexto en cero (`memset(&task->ctx, 0, ...)`).
 - Pone `task->ctx.ra = start_process`. Cuando el scheduler haga `context_switch` hacia este task por primera vez, `ret` saltará a `start_process()`.
 - Reserva la `kstack` con `alloc_page()` y deja `task->ctx.sp = kstack + PAGE_SIZE` (tope de la pila kernel).
 - Asigna `tid`, deja `pgtbl=kernel_pgtbl` provisoriamente, `pid=0`, `state=CREATED`.
2. Asignación de `pid`: `t->pid = ++last_pid`. Así el init queda con `pid 1` (es lo que imprime el `getpid` en el ejercicio 1).
3. `exec(t, path, 0)`: es el corazón de la creación del proceso.
 - a) Aloca `new_pgtbl` con `alloc_page()` y copia encima el `kernel_pgtbl` (`memcpy`). De esa forma todo proceso “ve” los mapeos del kernel para cuando entre en modo supervisor por un trap.
 - b) `load_program(new_pgtbl, path)`:
 - `efs_file(path)` devuelve el `struct file` con el binario.
 - Para cada `PAGE_SIZE` del binario aloca una página física con `alloc_page`, copia los bytes y mapea `va → pa` con flags `PAGE_R|W|X|U` (visible y ejecutable desde U-mode). La salida del ej. 1 muestra esto: "mapping user code va=00000000 to pa=8002e000".
 - c) Aloca la página del user-stack y la mapea en `PROC_MAX_VA - PAGE_SIZE` con `PAGE_R|W|U`. Así el stack usuario queda al tope del espacio de proceso.
 - d) `setup_main_args(task, ustack + PAGE_SIZE, args)`: empuja las cadenas y arma el array `char* argv[]`. Deja `argc` y `argv` en `a0/a1` del trap frame (a través de `put_argument`).
 - e) Inicializa el trap frame: `tf->sp = sp` (top del user-stack recién armado), `tf->pc = 0`. Esa `pc=0` corresponde al entry point `start()` del usuario (ver ej. 5).
 - f) Si el task ya era un proceso (re-exec), libera la memoria del page table anterior con `unmap_region(..., true)` y `free_page` del page table viejo.
 - g) `task->pgtbl = new_pgtbl`.
 - h) `task->ctx.sp = (size_t) tf`. Acá está el truco: cuando el scheduler haga `context_switch`, el `sp` del task quedará apuntando al trap frame en su `kstack`, y `u_trap_ret` va a leer todos los registros de ahí.

4. `t->state = RUNNABLE`. Recién ahora el scheduler puede elegir la tarea. La siguiente vez que el scheduler corra en alguna CPU, va a tomar este slot y `context_switch` hacia él.

Si algo falla, `exec()` libera lo allocateado y `create_process` retorna false. En el caso del `init`, `kernel_main()` no chequea el retorno: si fallara, EDOS quedaría sin procesos y solo correría el scheduler ocioso.

5. user.ld, entry point y mkefs.sh

(a) Linker script user.ld

```
ENTRY(start)

SECTIONS {
    . = 0;
    .text : { KEEP(*(.text.start)); *(.text .text.*); }
    .rodata : ALIGN(4) { *(.rodata .rodata.*); }
    .data : ALIGN(4) { *(.data .data.*); }
    .bss : ALIGN(4) { *(.bss .bss.* .sbss .sbss.*); }
}
```

El linker arranca la imagen en virtual address 0 (es lo que espera `exec()` cuando hace `tf->pc = 0`). El `KEEP` de `.text.start` fuerza que la primera función ubicada en `.text` sea la marcada con el atributo correspondiente; en este proyecto eso lo logra la posición física de `start()` como primer símbolo de `edoslib.c` enlazado primero por el Makefile del user (`edoslib.o usys.o init.c` en ese orden). El resultado es que la primera instrucción del `.text` efectivo es la del `start()`.

(b) Entry point

`ENTRY(start)` le dice al linker que el símbolo `start` es el punto de entrada, pero como el kernel ignora la cabecera ELF (el Makefile hace `objcopy -O binary init init.bin` y solo eso queda embebido en `efsfiles.c`), lo único que importa es que `start()` caiga en `va=0`. La función vive en `edoslib.c`:

```
void start(void) {
    exit(main());
}
```

O sea: cuando `exec()` arme el trap frame con `tf->pc = 0` y `sret` devuelva el control a U-mode, la CPU ejecuta `start()`, `start` llama al `main()` del programa, y cuando `main` retorna, `exit()` (`syscall 0`) le pasa el código de salida al kernel.

(c) Qué hace mkefs.sh

El script (lo invoca el Makefile del user con `./mkefs.sh init README`) arma `efsfiles.c` de la siguiente forma:

1. Crea un archivo temporal con un header y un `#include "efs.h"`.
2. Por cada nombre de archivo recibido, si existe `<nombre>.bin` (caso de los programas) usa `xxd -i <nombre>.bin` para volcar el binario como arreglo C; si no usa `xxd -i <nombre>` (caso del archivo de datos `README`). `xxd -i` genera dos símbolos: el arreglo de bytes `<nombre>_bin` (o `<nombre>`) y la longitud `<nombre>_bin_len`.

3. Con `sed`, reemplaza `unsigned int` por `const unsigned int` en las longitudes, para que el linker las ponga en `.rodata`.
4. Después abre la tabla `struct file efs_files_table[] = {` y, por cada archivo, emite una fila
`{"<nombre>", EFS_FILE_PROGRAM, <nombre>_bin, <nombre>_bin_len}` o
`{"<nombre>", EFS_FILE_DATA, <nombre>, <nombre>_len}` según haya `.bin` o no.
5. Cierra la tabla con un centinela `{0, 0, 0, 0}` y mueve el `efsfiles.c` al directorio del kernel.

En tiempo de compilación del kernel, `efs_file(name)` (`efs.c`) recorre esa tabla por `strcmp` y le devuelve a `load_program()` un puntero al binario y su largo. Así el kernel no necesita lectura de disco: el filesystem está en RAM, embebido en su `.rodata`.

6. Del scheduler a la primera instrucción del usuario

Recorrido completo de la primera vez que el `init` es elegido:

1. `scheduler()` (`task.c`) corre en CPU 0. Recorre `tasks[]`, toma el lock, encuentra el slot del `init` en estado `RUNNABLE`, lo pasa a `RUNNING`, asigna `next_task->ticks = QUANTUM`, deja `cpus_state[cpu].task = next_task` y llama:

```
context_switch(&cpus_state[cpu].ctx, &next_task->ctx);
```

2. `context_switch (arch.s)` guarda `ra` y `s0-s11` del scheduler en su propia pila (la pila del scheduler de esa CPU) y carga desde `next_task->ctx`:

```
sp <- next_task->ctx.sp (== kstack + PAGE_SIZE)
ra <- next_task->ctx.ra (== start_process)
```

Finalmente `ret` salta a `start_process`.

3. `start_process()` (`task.c`):

```
release(&task->lock);
return_to_user_mode();
```

Libera el lock que tenía el scheduler y va al retorno a usuario.

4. `return_to_user_mode()` (`trap.c`):

```
disable_interrupts();
set_u_trap_handler();           // stvec <- u_trap
set_kstack(kstack+PAGE_SIZE);   // sscratch <- kstack top
set_u_previous_mode();           // sstatus.SPP=0, SPIE=1
set_page_table(task->pgtbl);     // satp <- pa2satp(pgtbl)
u_trap_ret(task_trap_frame_address(task));
```

El `task_trap_frame_address` es exactamente el lugar de la `kstack` donde `exec()` dejó armado el trap frame inicial.

5. `u_trap_ret (trap.s)`:

```
mv sp, a0 # sp apunta al trap frame
lw a0, 30*4(sp)
csrw sepc, a0 # sepc <- tf->pc (== 0)
lw ra .. s11 # restaura los 29 registros guardados
```

```
lw sp, 29*4(sp) # sp <- tf->sp (top del user-stack)
sret
```

6. `sret` pasa a U-mode (porque `sstatus.SPP=0`), habilita interrupciones (`SPIE=1`) y carga `pc <- sepc = 0`. Como el `pgtbl` del proceso mapea `va=0` al código del `init`, la primera instrucción ejecutada en U-mode es la primera del `.text` del `init.bin`, que es la primera instrucción de `start()` (gracias a `user.ld` y al orden de linkeo del `user/Makefile`).
7. `start()` ejecuta `exit(main())`. Como `exit()` está en `usys.s` (con `ecall`), la primera vez que el proceso necesite interactuar con el kernel cae en `u_trap` por `SYSCALL` y se desencadena el camino del ej. 8.

7. Provocar un page fault en `init.c`

Cambié `init.c` (ver `entrega/init-ejercicio7.c`) para que el proceso intente escribir en una dirección no mapeada del espacio del proceso:

```
int main(void) {
    printf("EDOS init process: pid=%d!\n", getpid());

    printf("Intentando escribir en una direccion no mapeada "
           "(0xdeadbeef)...\n");
    volatile int *bad = (int *) 0xdeadbeef;
    *bad = 0x42;

    console_puts("Esta linea no deberia ejecutarse.\n");
    return 0;
}
```

Por qué dispara la falta:

- El proceso solo tiene mapeadas (con `PAGE_U`) las páginas de `[0, tamaño del programa)` y la página del stack en `[PROC_MAX_VA - PAGE_SIZE, PROC_MAX_VA)`, donde `PROC_MAX_VA` es `0x80000000`.
- `0xdeadbeef` está fuera de `PROC_MAX_VA`. Incluso si fuera menor, no está mapeada para U-mode.
- El store dispara una excepción `STORE_PAGE_FAULT` (`scause=0x0F`). El handler `u_trap` llama a `user_trap()`, que cae en la rama de “page fault. Killing task...” y deja al task con `killed=true`. En la salida del scheduler aparece, después, `free_resources`.

Verificación (extracto de `entrega/salida-ejercicio7.txt`):

```
EDOS init process: pid=1!
Intentando escribir en una direccion no mapeada (0xdeadbeef)...
Task init in CPU 0 page fault. Killing task...
Freeing task init resources...
```

La línea “Esta linea no deberia ejecutarse” no aparece, como esperaba.

8. Camino completo del syscall `getpid()`

Paso a paso, desde el `call()` en el código de usuario hasta el `ret` en el wrapper:

1. Código de usuario (`init.c`): “`printf(... getpid() ...)`” se resuelve al símbolo externo `getpid` declarado en `edoslib.h`.

2. Stub assembler (user/usys.s):

```
getpid:
    li a7, 1      # SYS_GETPID
    ecall
    ret
```

Carga el número de syscall en `a7` y ejecuta `ecall`. La ABI de RISC-V deja eventuales argumentos en `a0..a6`; `getpid()` no tiene.

3. `ecall` genera una excepción con `scause = 0x8` (ENVIRONMENT CALL FROM U-MODE) y `sepc = pc` del `ecall`.
4. `u_trap (trap.s)`: swap `sp <-> sscratch` (pasa a usar la `kstack` del proceso), reserva `31*4` bytes para el `struct trap_frame` y guarda `ra`, `gp`, `t0-t6`, `a0-a7`, `s0-s11`, el `sp` del usuario (leído de `sscratch`) y `sepc` (en `tf->pc`). Después `call user_trap()`.
5. `user_trap (trap.c)`:
 - `set_page_table(kernel_pgtbl)`: pasa al page table del kernel mientras corre el handler.
 - `set_k_trap_handler()`: por si dentro del syscall ocurre otro trap (en S-mode).
 - `case SYSCALL`:

```
enable_interrupts();
syscall(task);
skip_trap_instruction(task_trap_frame_address(task));
```

`skip_trap_instruction` hace `tf->pc += 4`, que es lo que dejará el `sepc` tras `u_trap_ret` apuntando a la instrucción siguiente al `ecall`.

6. `syscall() (syscall.c)`:
 - `n = syscall_number(tf) = tf->a7 = 1`.
 - Dispatcher: `syscalls_table[1] = sys_getpid`.
 - `sys_getpid(task)` retorna `task->pid` (el pid del proceso).
 - `syscall_put_result(tf, result)` deja el valor en `tf->a0`.
7. `user_trap` termina llamando a `return_to_user_mode()` al final (y un `panic` de seguridad por si esa función retornara), así que el flujo no vuelve por donde vino sino que rearma el retorno a usuario.
8. `return_to_user_mode()` rearma el entorno (`set_u_trap_handler`, `set_kstack`, `set_u_previous_mode`, `set_page_table` al `pgtbl` del proceso) y llama a `u_trap_ret()` pasándole la dirección del trap frame (`task_trap_frame_address`).
9. `u_trap_ret (trap.s)` restaura todos los registros (incluyendo `a0` con el resultado del syscall), pone `sepc <- tf->pc` (que ya quedó apuntando a la instrucción siguiente al `ecall`), y `sret` pasa de nuevo a U-mode.
10. La CPU ejecuta la instrucción siguiente al `ecall`, que es el `ret` de `getpid`. `a0` ya tiene el pid, así que el llamador del usuario recibe el valor de retorno como una función C común.

9. Agregar el syscall time()

Decidí implementar `time()` devolviendo el contador global de ticks del kernel (`volatile unsigned int ticks` en `task.c`). Es lo natural: ese `ticks` lo incrementa `inc_ticks()` en cada timer interrupt y ya hay una función `get_ticks()` que lo lee tomando el `ticks_lock`.

Cambios realizados (todos en entrega/):

1) syscall.h

```
#define SYS_TIME      6
#define SYSCALLS      7
```

2) **syscall.c** — agregué `sys_time` y lo sumé al final de `syscalls_table[]` (que ahora tiene 7 entradas, igual a `SYSCALLS`):

```
int sys_time(struct task *task) {
    (void) task;
    return (int) get_ticks();
}
```

3) user/edolib.h

```
extern int time(void);
```

4) user/usys.s

```
.global time
time:
    li a7, 6      # SYS_TIME
    ecall
    ret
```

5) **user/init.c** — modifiqué `main()` para imprimir el valor de `time()` varias veces, durmiendo 2 ticks entre llamadas:

```
for (int i = 0; i < 5; i++) {
    printf("init: time = %d ticks\n", time());
    sleep(2);
}
printf("init: time final = %d ticks. Finishing...\n", time());
```

El uso de `get_ticks()` (con su spinlock `ticks_lock`) en lugar de leer `ticks` directamente evita una race con `inc_ticks()` en el caso multi-CPU. Devuelvo `int` para mantener la firma compatible con el dispatcher (`typedef int (*syscall_f)(struct task *)`).

Verificación (extracto de `entrega/salida-ejercicio9.txt`):

```
EDOS init process: pid=1!
init: time = 0 ticks
pid 1 going to sleep.
Task pid=1 awake.
init: time = 2 ticks
pid 1 going to sleep.
Task pid=1 awake.
init: time = 4 ticks
pid 1 going to sleep.
Task pid=1 awake.
init: time = 6 ticks
pid 1 going to sleep.
Task pid=1 awake.
init: time = 8 ticks
pid 1 going to sleep.
Task pid=1 awake.
```

```
init: time final = 10 ticks. Finishing...  
Freeing task init resources...
```

El contador sube de a 2 entre cada print porque entre llamada y llamada el proceso duerme 2 ticks; el último `time()` lee 10 porque hubo 5 iteraciones de `sleep(2)` además del primer print. Eso valida que el syscall está bien numerado, el stub pasa por la tabla de syscalls y el resultado viaja de `a0` hasta el llamador en user.